

14. CLOB Operation

A CLOB (Character Large Object) is used to store large amounts of text data in a database. It is ideal for storing long documents, JSON, XML, or other textual content. In JDBC, CLOB data can be inserted and retrieved using PreparedStatement and ResultSet.

Database Table Creation

Run the following SQL query to create the required table:

```
CREATE TABLE ClobExample (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255),  
    text_data CLOB  
);
```

Syntax:

- 1) public void setBinaryStream(int paramIndex,InputStream stream)throws SQLException
- 2) public void setBinaryStream(int paramIndex,InputStream stream,long length)throws SQLException

Example to store file in database

```
package com.lokesh;  
  
import java.io.*;  
import java.sql.*;  
  
public class ClobStoreExample {  
  
    public static void main(String[] args) {  
        String jdbcUrl = "jdbc:mysql://localhost:3306/organization";  
        String username = "root";  
        String password = "root";  
  
        Connection connection = null;  
        PreparedStatement preparedStatement = null;  
  
        try {  
            // Establish connection  
            connection = DriverManager.getConnection(jdbcUrl, username, password);  
  
            // Read the text file from the resources folder  
            File file = new File("src/main/resources/store_file.txt");  
            FileReader reader = new FileReader(file);  
  
            // Prepare the SQL statement  
            String sql = "INSERT INTO ClobExample (name, text_data) VALUES (?, ?)";  
            preparedStatement = connection.prepareStatement(sql);  
            preparedStatement.setString(1, file.getName());  
            preparedStatement.setCharacterStream(2, reader, file.length());  
  
            // Execute the query  
            int rowsInserted = preparedStatement.executeUpdate();  
            if (rowsInserted > 0) {  
                System.out.println("Text successfully stored as CLOB.");  
            }  
        }  
    }  
}
```

```

    }

    reader.close();
} catch (Exception e) {
    e.printStackTrace();
} finally {
    // Close resources
    try {
        if (preparedStatement != null) preparedStatement.close();
        if (connection != null) connection.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}
}
}

```

Output:

Text successfully stored as CLOB.

The getClob() method of PreparedStatement is used to get file information from the database.

Syntax of getClob method

```
public Clob getClob(int columnIndex){}
```

Example to retrieve file in database

```

package com.lokesh;
import java.io.*;
import java.sql.*;

public class ClobRetrieveExample {

    public static void main(String[] args) {
        String jdbcUrl = "jdbc:mysql://localhost:3306/organization";
        String username = "root";
        String password = "root";

        Connection connection = null;
        PreparedStatement preparedStatement = null;
        ResultSet resultSet = null;

        try {
            // Establish connection
            connection = DriverManager.getConnection(jdbcUrl, username, password);

            // Prepare the SQL query
            String sql = "SELECT name, text_data FROM ClobExample WHERE id = ?";
            preparedStatement = connection.prepareStatement(sql);
            preparedStatement.setInt(1, 1); // Assuming the ID of the text to retrieve is 1

            // Execute the query
            resultSet = preparedStatement.executeQuery();

            if (resultSet.next()) {
                String fileName = resultSet.getString("name");
                Reader reader = resultSet.getCharacterStream("text_data");
            }
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            if (resultSet != null) resultSet.close();
            if (preparedStatement != null) preparedStatement.close();
            if (connection != null) connection.close();
        }
    }
}

```

```
// Save the text file to the resources folder
File outputFile = new File("src/main/resources/retrieved_" + fileName);
FileWriter writer = new FileWriter(outputFile);

int character;
while ((character = reader.read()) != -1) {
    writer.write(character);
}

System.out.println("Text successfully retrieved and saved to resources
                    folder.");

writer.close();
reader.close();
}
} catch (Exception e) {
    e.printStackTrace();
} finally {
    // Close resources
    try {
        if (resultSet != null) resultSet.close();
        if (preparedStatement != null) preparedStatement.close();
        if (connection != null) connection.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}
```

Output:

Text successfully retrieved and saved to resources folder.